

Análise e Projeto de Software Orientado a Objetos

Engenharia de Requisitos: Casos de Uso, Diagramas UML e Validação

Leonardo S. Victorio

Faculdade de Computação - UFMS

2026/1



- 1 Introdução aos Casos de Uso
- 2 Diagrama de Casos de Uso
- 3 Descrição Textual dos Casos de Uso
- 4 Cenários nos Casos de Uso
- 5 Casos de Uso na Prática
- 6 O Documento de Requisitos de Software
- 7 Validação de Requisitos
- 8 Construindo Diagramas de Casos de Uso
- 9 Conclusão



O que são Casos de Uso?

Definição (Sommerville)

Casos de uso são uma maneira de descrever as interações entre usuários e um sistema usando um **modelo gráfico** e um **texto estruturado**.

- Foram introduzidos pela primeira vez no método **Objectory** (JACOBSON et al., 1993).
- Tornaram-se uma característica fundamental da **UML** (Unified Modeling Language).
- Identificam os **atores** envolvidos em uma interação e nomeiam o **tipo de interação**.
- Podem ser complementados por diagramas de sequência ou de máquina de estados.



Método Objectory (1993)

- Criado por Ivar Jacobson.
- Primeiro método orientado a objetos a utilizar casos de uso sistematicamente.
- Foco na interação ator-sistema.

UML (Padrão Atual)

- A UML padronizou os casos de uso como artefato central.
- Diagrama de Casos de Uso é um dos 14 tipos de diagramas UML.
- Padrão amplamente adotado pela indústria.



Por que usar Casos de Uso?

- **Comunicação:** Facilitam o diálogo entre engenheiros e stakeholders não técnicos.
- **Escopo:** O conjunto de casos de uso representa **todas as interações possíveis** descritas nos requisitos de sistema.
- **Base para testes:** Cada caso de uso pode derivar casos de teste para validação.
- **Documentação:** Servem como registro das funcionalidades esperadas do sistema.

Observação de Sommerville

Casos de uso são mais úteis no **projeto de sistemas** do que propriamente na fase inicial de elicitação de requisitos, pois stakeholders muitas vezes não compreendem o termo.



Casos de Uso e a Engenharia de Requisitos

- A **UML** é um padrão para modelagem orientada a objetos.
- Os casos de uso e a elicitação baseada neles são utilizados no processo de engenharia de requisitos.
- No entanto, na prática, eles são muitas vezes **refinados demais** para a discussão inicial de requisitos.

Posição do Sommerville

- Stakeholders não compreendem o termo *caso de uso*.
- Não acham útil o modelo gráfico isoladamente.
- Muitas vezes não estão interessados em uma descrição detalhada de cada interação.



A Forma Mais Simples de um Caso de Uso

Em sua forma mais simples, um caso de uso:

- 1 **Identifica os atores** envolvidos na interação.
- 2 **Nomeia o tipo de interação** que ocorre.
- 3 **Adiciona informações descritivas** sobre essa interação.

Essas informações podem ser:

- Uma **descrição textual** da interação.
- Um ou mais **modelos gráficos** (diagramas de sequência, máquina de estados).



Componentes Básicos do Diagrama

Os casos de uso são documentados por meio de um **diagrama de casos de uso de alto nível**:

Atores Representados como “bonecos palito”. Podem ser seres humanos ou outros sistemas.

Casos de Uso Representados como **elipses nomeadas**. Cada elipse indica uma classe de interação.

Linhas de Associação Fazem a ligação entre atores e interações.

Pontas de Seta Opcionais; indicam quem *inicia* a interação.

Sistema Retângulo que delimita a fronteira do sistema.



O que é um Ator?

Definição

Um **ator** é qualquer entidade — humana ou sistema externo — que interage diretamente com o sistema para alcançar um objetivo.

Atores Humanos

- Recepcionista
- Médico
- Gerente
- Profissional de enfermagem

Atores Sistêmicos

- Sistema bancário externo
- Sistema de autenticação
- Serviço de e-mail
- Banco de dados legado



O sistema Mentcare (sistema de informações de saúde mental) possui os seguintes atores e casos de uso:

Atores:

- Recepcionista
- Profissional de enfermagem
- Médico
- Gerente

Casos de Uso:

- Registrar o paciente
- Visualizar informações pessoais
- Visualizar registro
- Editar registro
- Realizar discussão de caso
- Exportar estatísticas
- Gerar relatório



Leitura do Diagrama: Relacionamentos

- A **linha simples** entre ator e caso de uso indica uma **associação** — o ator participa daquele caso de uso.
- Um ator pode estar associado a **múltiplos** casos de uso.
- Um caso de uso pode envolver **múltiplos** atores.

Exemplo do Mentcare

- **Recepcionista** → Registrar paciente, Visualizar informações pessoais.
- **Médico** → Visualizar registro, Editar registro, Realizar discussão de caso.
- **Profissional de enfermagem** → Visualizar registro, Editar registro, Realizar discussão de caso.
- **Gerente** → Exportar estatísticas, Gerar relatório.

Além da associação simples, a UML define relações entre casos de uso:

«**include**» Um caso de uso **inclui** obrigatoriamente o comportamento de outro. Usado para fatorar passos comuns.

«**extend**» Um caso de uso **estende** outro com comportamento opcional ou condicional. Usado para variações.

Generalização Um caso de uso filho herda o comportamento do pai e pode sobrescrevê-lo.

Exemplo Prático

Editar Registro «include» Autenticar Usuário — sempre que alguém edita um registro, deve estar autenticado.

- No diagrama, um **retângulo** delimita o sistema sendo modelado.
- Casos de uso ficam **dentro** do retângulo (são responsabilidade do sistema).
- Atores ficam **fora** do retângulo (são entidades externas).
- A fronteira define claramente o **escopo** do sistema: o que está dentro e o que está fora.

Importância do Escopo

Definir corretamente a fronteira evita a inclusão de funcionalidades além do escopo e ajuda a alinhar expectativas com os stakeholders.



Alto Nível vs. Detalhado

- O **diagrama de alto nível** mostra apenas as principais interações — adequado para comunicação com clientes.
 - **Diagramas detalhados** incluem relações *include/extend* e especificam subfluxos.
-
- O nível de granularidade deve ser **proporcional ao estágio** do desenvolvimento.
 - Muito detalhe prematuramente pode confundir mais do que ajudar.
 - O conjunto de casos de uso representa **todas as interações possíveis** que serão descritas nos requisitos de sistema.



Princípio Fundamental

Cada caso de uso deve ser documentado com uma **descrição textual**, que pode ser ligada a outros modelos UML para compor um cenário mais detalhado.

- O diagrama gráfico é apenas a **visão geral** — sozinho, não é suficiente.
- A descrição textual fornece o **contexto**, as **pré-condições**, os **fluxos** e as **pós-condições**.
- Diferentes níveis de detalhe são usados em diferentes fases do projeto.



Exemplo Real: “Realizar Discussão de Caso”

Descrição Resumida (Sommerville, 2011)

Realizar discussão de caso permite que dois ou mais médicos, trabalhando em consultórios diferentes, vejam o registro do mesmo paciente ao mesmo tempo. Um médico inicia a discussão do caso de um paciente escolhendo as pessoas envolvidas em um menu suspenso de médicos que estão on-line. O registro do paciente é exibido em suas telas, mas apenas o médico que iniciou a consulta pode editar o registro. Além disso, cria-se um chat para ajudar a coordenar as ações. Presume-se que uma chamada telefônica ou comunicação por voz possa ser providenciada separadamente.



Analisando a Descrição: Elementos Presentes

Ao analisar a descrição do caso “Realizar Discussão de Caso”, identificamos:

Atores Dois ou mais médicos (ator principal: médico que inicia).

Pré-condição implícita Médicos devem estar autenticados e on-line.

Fluxo principal Escolher participantes → Exibir registro → Colaborar.

Restrição Apenas o médico iniciador pode editar o registro.

Funcionalidade auxiliar Chat integrado para coordenação.

Fora do escopo Comunicação por voz (presumida como externa).



Campos Típicos

- **Nome:** Verbo + substantivo (ex: “Registrar Paciente”).
- **ID:** Identificador único (ex: UC-05).
- **Ator(es) principal(is):** Quem inicia a interação.
- **Ator(es) secundário(s):** Quem participa, mas não inicia.
- **Pré-condições:** Estado do sistema antes da execução.
- **Fluxo principal (caminho feliz):** Sequência normal de eventos.
- **Fluxos alternativos/de exceção:** O que acontece se algo der errado.
- **Pós-condições:** Estado do sistema após a execução.



Caso de Uso: Registrar Paciente (UC-01)

- **Ator principal:** Recepcionista.
- **Pré-condição:** Recepcionista autenticada no sistema.
- **Fluxo principal:**
 - 1 Recepcionista seleciona “Novo Paciente”.
 - 2 Sistema exibe formulário de cadastro.
 - 3 Recepcionista preenche os dados pessoais.
 - 4 Sistema valida os dados e salva o registro.
 - 5 Sistema exibe confirmação com número do paciente.
- **Fluxo alternativo:** Dados inválidos → sistema exibe mensagem de erro e permite correção.
- **Pós-condição:** Paciente registrado; registro disponível para médicos.



Descrição Resumida

- Parágrafo narrativo curto.
- Adequada para fases iniciais.
- Facilita comunicação com stakeholders não técnicos.
- Usada em diagramas de alto nível.

Descrição Detalhada

- Formulário estruturado completo.
- Adequada para fases de projeto.
- Inclui todos os fluxos e exceções.
- Usada como base para casos de teste.

Boas Práticas na Descrição Textual

- **Nome no infinitivo:** “Registrar”, “Visualizar”, “Gerar” — ação do ponto de vista do ator.
- **Evitar soluções técnicas:** Descreva *o que* acontece, não *como* será implementado.
- **Voz ativa e clara:** “O médico seleciona...” e não “O registro é selecionado pelo médico...”.
- **Um passo por vez:** Cada etapa do fluxo deve ser atômica e verificável.
- **Cobrir as exceções:** Identificar o que acontece quando algo foge do fluxo normal.

Atenção

Casos de uso descrevem **comportamento externo** do sistema — o que o usuário observa — e não a lógica interna de implementação.

Ligação entre Descrição Textual e Outros Modelos UML

A descrição textual pode ser complementada por:

Diagrama de Sequência Mostra a troca de mensagens entre objetos ao longo do tempo para executar o caso de uso.

Diagrama de Atividades Representa o fluxo de controle entre as etapas do caso de uso.

Diagrama de Máquina de Estados Mostra como o sistema muda de estado durante a execução.

Na Prática

Para casos de uso simples, a descrição textual basta. Para casos complexos com muitos fluxos alternativos, complementar com diagramas reduz ambiguidades.



O que é um Cenário?

Definição

Um **cenário** é uma instância de execução de um caso de uso: uma sequência específica de eventos que ocorre durante uma interação particular entre um ator e o sistema.

- Cenários são **concretos e situacionais** — descrevem uma situação específica.
- Um caso de uso pode englobar **múltiplos cenários** (normal, alternativo, exceção).
- São úteis para **comunicar exemplos** reais com os stakeholders.
- Ajudam a identificar fluxos alternativos que poderiam ser esquecidos.



Duas Visões sobre a Relação Caso de Uso — Cenário

Existe uma discussão na literatura sobre como relacionar casos de uso e cenários:

Visão 1: 1 Caso de Uso = 1 Cenário

- Cada caso de uso é um **cenário único e detalhado**.
- Abordagem mais simples e direta.
- Mais fácil de gerenciar em projetos pequenos.

Visão 2: Stevens & Pooley (2006)

- Cada caso de uso inclui um **conjunto relacionado de cenários**.
- Um cenário por caminho do caso de uso.
- Mais adequado para casos de uso complexos.

Sommerville

Na prática, **dá para usá-los de ambas as formas**, dependendo da complexidade do sistema e do estilo da equipe.

❶ Cenário principal (caminho feliz):

A sequência normal de eventos quando tudo corre como esperado.
Ex.: Médico acessa o sistema, localiza o paciente e edita o registro com sucesso.

❷ Cenários alternativos:

Variações válidas do fluxo principal que levam ao mesmo resultado.
Ex.: Médico não encontra o paciente pelo nome e busca pelo CPF.

❸ Cenários de exceção:

Situações de erro que interrompem o fluxo normal.
Ex.: Sessão expira durante a edição — sistema solicita reautenticação.



Cenários como Ferramenta de Elicitação

- Cenários são altamente eficazes para **elicitar requisitos**, pois são concretos e compreensíveis por leigos.
- Em vez de perguntar “Quais são os requisitos do sistema?”, o analista pergunta:
“O que você faz quando precisa registrar um paciente novo? Me conte passo a passo.”
- A resposta do stakeholder é, essencialmente, um **cenário** que pode ser formalizado em um caso de uso.

Vantagem

As pessoas se lembram melhor de **situações concretas** do que de abstrações. Perguntar sobre cenários extrai informações mais ricas e precisas.



Exemplo: Cenários do Caso de Uso “Editar Registro”

Cenário 1: Caminho Feliz

Médico autentica-se. Busca o paciente pelo número de matrícula. Localiza o registro. Atualiza as informações de diagnóstico. Sistema salva e exibe confirmação.

Cenário 2: Paciente não Localizado

Médico autentica-se. Busca pelo nome e não encontra. Sistema sugere busca por CPF. Médico informa CPF. Paciente é localizado. Médico edita o registro.

Cenário 3: Permissão Insuficiente

Profissional de enfermagem tenta editar um campo restrito. Sistema exibe mensagem: “Você não tem permissão para editar este campo.” Ação é bloqueada.



O processo de formalização de cenários segue uma progressão:

- ❶ **Narrativa informal:** Descrição em linguagem natural coletada com o stakeholder.
- ❷ **Descrição estruturada:** Preenchimento do formulário de caso de uso (pré-condições, fluxos, pós-condições).
- ❸ **Diagrama de casos de uso:** Representação gráfica UML das interações identificadas.
- ❹ **Modelos complementares:** Diagramas de sequência para detalhar os fluxos mais complexos.

Atenção

A narrativa não substitui a formalização, mas é o ponto de partida. Sem ela, o diagrama pode ser criado sem compreensão real do domínio.

Casos de Uso e Especificações Tabulares

Em alguns casos, especialmente quando há múltiplas condições, **tabelas** complementam as descrições textuais:

Condição	Ação do Sistema
Dados do paciente válidos	Salvar registro e confirmar
CPF já cadastrado	Exibir alerta de duplicidade
Campo obrigatório em branco	Destacar campo e bloquear salvamento
Sessão expirada	Redirecionar para tela de login

Tabelas são úteis quando há **série de situações alternativas** e é preciso descrever as ações para cada uma delas.



Casos de Uso no Processo de Desenvolvimento

Onde se Encaixam?

Os casos de uso são utilizados em diferentes fases do desenvolvimento orientado a objetos:

- ❶ **Elicitação de Requisitos:** Para capturar interações com stakeholders.
- ❷ **Análise:** Para identificar as classes e objetos do domínio.
- ❸ **Projeto (Design):** Para guiar o design das interfaces e a distribuição de responsabilidades.
- ❹ **Testes:** Como base para casos de teste de aceitação.

Sommerville

Os casos de uso são mais úteis no **projeto de sistemas** do que na engenharia de requisitos propriamente dita.

O Desafio com Stakeholders

Ao utilizar casos de uso diretamente com stakeholders não técnicos:

- Os stakeholders frequentemente **não compreendem** o termo “caso de uso”.
- O modelo gráfico (diagrama) nem sempre é considerado **útil ou intuitivo** por eles.
- Muitas vezes, não estão interessados em uma descrição detalhada de **cada interação** do sistema.

Estratégia Recomendada

Utilizar cenários narrativos e linguagem natural na fase inicial com stakeholders. Formalizar em casos de uso UML posteriormente com a equipe técnica.



Casos de Uso como Base para o Projeto OO

- No capítulo 5 de Sommerville, casos de uso são utilizados com outros modelos para **documentar o projeto (design)**.
- Cada caso de uso pode guiar a identificação de:
 - **Classes** do domínio (substantivos nos fluxos).
 - **Métodos** das classes (verbos nos fluxos).
 - **Responsabilidades** e colaborações entre objetos.
- O diagrama de sequência derivado do caso de uso mostra como os objetos **trocam mensagens** para realizar a funcionalidade.



- ④ **Ler** a descrição do sistema ou narrativa do stakeholder.
- ② **Identificar os atores:** Quem interage com o sistema? Quem se beneficia?
- ③ **Identificar os casos de uso:** Quais são os objetivos dos atores? Quais serviços o sistema oferece?
- ④ **Definir o escopo:** O que está dentro e fora do sistema?
- ⑤ **Descrever textualmente** cada caso de uso com pré-condições, fluxo e pós-condições.
- ⑥ **Construir o diagrama UML** representando os relacionamentos.
- ⑦ **Validar** com os stakeholders.

Exercício de Leitura: Identificando Atores e Casos de Uso

Texto de exemplo (sistema de emissão de bilhetes):

Uma máquina de emitir bilhetes vende passagens de trem. Os usuários escolhem seu destino e fornecem um cartão de crédito e um número de identificação pessoal. A passagem de trem é emitida e o cartão de crédito dos usuários é cobrado. Quando os usuários pressionam o botão de iniciar, ativam um menu de possíveis destinos, junto com uma mensagem para a escolha de um destino e do tipo de bilhete necessário.

Identifique:

- **Atores:**



Exercício de Leitura: Identificando Atores e Casos de Uso

Texto de exemplo (sistema de emissão de bilhetes):

Uma máquina de emitir bilhetes vende passagens de trem. Os usuários escolhem seu destino e fornecem um cartão de crédito e um número de identificação pessoal. A passagem de trem é emitida e o cartão de crédito dos usuários é cobrado. Quando os usuários pressionam o botão de iniciar, ativam um menu de possíveis destinos, junto com uma mensagem para a escolha de um destino e do tipo de bilhete necessário.

Identifique:

- **Atores:** Usuário (passageiro), Sistema bancário (cartão de crédito).
- **Casos de uso:**



Exercício de Leitura: Identificando Atores e Casos de Uso

Texto de exemplo (sistema de emissão de bilhetes):

Uma máquina de emitir bilhetes vende passagens de trem. Os usuários escolhem seu destino e fornecem um cartão de crédito e um número de identificação pessoal. A passagem de trem é emitida e o cartão de crédito dos usuários é cobrado. Quando os usuários pressionam o botão de iniciar, ativam um menu de possíveis destinos, junto com uma mensagem para a escolha de um destino e do tipo de bilhete necessário.

Identifique:

- **Atores:** Usuário (passageiro), Sistema bancário (cartão de crédito).
- **Casos de uso:** Selecionar destino, Fornecer pagamento, Emitir bilhete.



Ambiguidades e Omissões nos Requisitos

Ao analisar o texto do sistema de bilhetes, percebemos problemas típicos:

- **Ambiguidade:** “Número de identificação pessoal” — PIN do cartão ou CPF do usuário?
- **Omissão:** O que acontece se o cartão for recusado?
- **Omissão:** O que acontece se o destino selecionado estiver indisponível?
- **Omissão:** É possível cancelar a operação no meio do processo?
- **Ambiguidade:** “Tipo de bilhete necessário” — ida e volta? Classe?

Conclusão

Construir casos de uso formais força a **identificação de ambiguidades e omissões** que passariam despercebidas em descrições informais.

Casos de Uso e Requisitos Não Funcionais

- Casos de uso descrevem **interações funcionais** — o que o sistema faz.
- Requisitos Não Funcionais (RNFs) **não aparecem** diretamente nos casos de uso.
- Os RNFs se aplicam **transversalmente** a múltiplos casos de uso.

Exemplo

- **Caso de Uso:** Registrar Paciente (descreve o fluxo).
- **RNF associado:** O registro deve ser salvo em no máximo 2 segundos (desempenho).
- **RNF associado:** Os dados devem ser criptografados em repouso (segurança).

Anotar os RNFs relevantes diretamente na documentação de cada caso de uso é uma boa prática.



Casos de Uso como Contrato entre Cliente e Desenvolvedor

- Os casos de uso funcionam como um **contrato informal**: o que o sistema promete fazer.
- A validação por stakeholders garante que o entendimento está **alinhado antes do desenvolvimento**.
- Alterações nos casos de uso após o início do desenvolvimento geram **retrabalho e custo**.

Boas Práticas de Gerenciamento

- Numerar e versionar cada caso de uso.
- Registrar quem aprovou cada versão.
- Associar casos de uso a requisitos funcionais numerados.
- Manter rastreabilidade entre caso de uso → módulo → teste.

O que é o Documento de Requisitos?

Definição

O **Documento de Requisitos de Software** (às vezes chamado de Especificação de Requisitos de Software ou **ERS/SRS**) é uma declaração oficial do que os desenvolvedores do sistema devem implementar.

- Pode incluir os **requisitos de usuário** e uma **especificação detalhada** dos requisitos de sistema.
- Às vezes, requisitos de usuário e de sistema são integrados em uma descrição única.
- Em outros casos, os requisitos de usuário são descritos em um capítulo introdutório.



Quando o Documento é Essencial?

Os documentos de requisito são **essenciais** quando:

- Os sistemas têm seu **desenvolvimento terceirizado**.
- **Times diferentes** desenvolvem partes diferentes do sistema.
- Uma **análise detalhada** dos requisitos é obrigatória (sistemas críticos, regulados).

Em outras circunstâncias, como o desenvolvimento de um **produto de software interno** ou de um **sistema de negócio ágil**, um documento de requisitos detalhado pode não ser necessário.

Métodos Ágeis

Em vez de um documento formal, as abordagens ágeis coletam requisitos incrementalmente em **histórias de usuário** em cartões ou lousas.



Usuários do Documento de Requisitos

O documento tem um conjunto **diverso** de usuários:

Clientes do sistema Especificam os requisitos e os leem para conferir se satisfazem suas necessidades. Especificam mudanças nos requisitos.

Gerentes Usam o documento para planejar uma proposta para o sistema e planejar o processo de desenvolvimento.

Engenheiros de sistema Usam os requisitos para compreender qual sistema deve ser desenvolvido.

Engenheiros de teste Usam os requisitos para desenvolver testes de validação do sistema.

Engenheiros de manutenção Usam os requisitos para entender o sistema e as relações entre suas partes.



Estrutura do Documento de Requisitos (IEEE)

Capítulo	Descrição
Prefácio	Histórico de versões e fundamentação
Introdução	Necessidade do sistema e objetivos de negócio
Glossário	Termos técnicos utilizados
Req. de usuário	Serviços fornecidos ao usuário (RF e RNF)
Arq. do sistema	Visão geral e distribuição de funções
Esp. de sistema	Requisitos funcionais e não funcionais detalhados
Modelos do sistema	Diagramas UML
Evolução	Mudanças previstas no sistema
Apêndices	Hardware, banco de dados etc.
Índice	Índice alfabético, de diagramas e funções



Prefácio

- Define o público-alvo do documento.
- Histórico de versões.
- Fundamentação para cada nova versão.
- Resumo das mudanças feitas.

Introdução

- Descreve a **necessidade** do sistema.
- Resume as funções do sistema.
- Explica como o sistema trabalha com outros sistemas.
- Como o sistema se encaixa nos **objetivos de negócio**.

Modelos do Sistema

Inclui modelos gráficos mostrando as relações entre os componentes do sistema e entre o sistema e seu ambiente. Exemplos:

- Modelos de objeto (diagramas de classes).
- Modelos de fluxo de dados.
- Modelos semânticos de dados.
- Diagramas de casos de uso e de sequência.

Evolução do Sistema

Descreve os pressupostos fundamentais e mudanças previstas em virtude da **evolução do hardware**, das necessidades dos usuários etc. Ajuda projetistas a evitar decisões que restringiriam futuras mudanças.

Nível de Detalhe do Documento

O nível de detalhe depende do tipo de sistema e do processo de desenvolvimento:

- **Sistemas críticos (safety/security):** Requisitos muito detalhados, pois erros podem causar riscos graves.
- **Desenvolvimento terceirizado:** Especificações detalhadas e precisas são obrigatórias para evitar disputas contratuais.
- **Desenvolvimento interno iterativo:** Pode ser menos detalhado; ambiguidades são resolvidas durante o desenvolvimento.

Documento longo $\neq$ Documento bom

A diversidade dos usuários significa que o documento deve **ser acordado por todos**. Inclua uma tabela de conteúdo abrangente e um índice para facilitar a navegação.



Principais Organizações Padronizadoras

- **IEEE** (Institute of Electrical and Electronics Engineers) — desenvolveu o padrão IEEE 830 para estrutura de documentos de requisitos.
- **Departamento de Defesa dos EUA** — define padrões para sistemas militares com longa vida útil.
- Esses padrões são **genéricos** e podem ser adaptados a usos específicos.
- O padrão IEEE é mais adequado para sistemas como **comando e controle militar**, desenvolvidos por um grupo de organizações ao longo de muitos anos.
- Organizações podem criar **padrões internos** baseados nesses padrões genéricos.



Documento de Requisitos vs. Métodos Ágeis

Desenvolvimento Tradicional

- Documento formal completo.
- Aprovado antes do desenvolvimento.
- Serve de contrato entre cliente e fornecedor.
- Essencial para sistemas críticos e terceirizados.

Desenvolvimento Ágil

- Requisitos em cartões (histórias de usuário).
- Incrementais e priorizados pelo cliente.
- Documento formal pode ficar obsoleto rapidamente.
- Ainda útil para requisitos de dependabilidade.

Recomendação de Sommerville

Mesmo em projetos ágeis, é útil manter um **documento resumido** que defina o negócio e os requisitos de dependabilidade — é fácil esquecê-los ao focar nos requisitos funcionais de cada sprint.

O que é Validação de Requisitos?

Definição

A **validação de requisitos** é o processo de conferir se os requisitos definem o sistema que o cliente **realmente quer**. Ele se sobrepõe à elicitação e à análise, pois é voltado para **encontrar problemas** antes que eles se tornem caros.

- Erros em um documento de requisitos podem levar a **grandes custos de retrabalho** quando descobertos durante o desenvolvimento.
- O custo de corrigir um problema nos requisitos com uma alteração no sistema é muito **maior do que corrigir erros de projeto ou de código**.
- Uma mudança nos requisitos significa que o projeto e a implementação **também deverão ser modificados**.



Por que a Validação é Criticamente Importante?

O Custo Cresce com o Tempo

Estudos clássicos de engenharia de software mostram que o custo de corrigir um defeito cresce exponencialmente conforme avançamos nas fases do desenvolvimento:

- Corrigir na **fase de requisitos**: custo 1x.
- Corrigir na **fase de projeto**: custo 5x.
- Corrigir na **fase de implementação**: custo 10x.
- Corrigir após a **entrega**: custo 100x.



As Cinco Conferências da Validação

Durante o processo de validação de requisitos, cinco tipos de conferências devem ser executados:

- ❶ **Conferência da validade:** Os requisitos refletem as reais necessidades dos usuários do sistema?
- ❷ **Conferência da consistência:** Os requisitos no documento não entram em conflito entre si?
- ❸ **Conferência da completude:** O documento inclui todos os serviços e restrições pretendidos?
- ❹ **Conferência do realismo:** Os requisitos podem ser implementados com o orçamento e tecnologia disponíveis?
- ❺ **Verificabilidade:** É possível escrever um conjunto de testes que demonstre que o requisito foi satisfeito?



Os Três Elementos Visuais do Diagrama

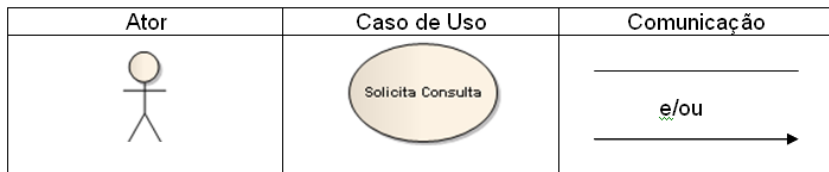
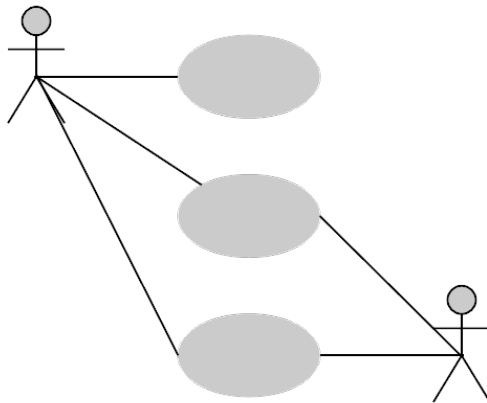


Diagrama Básico: Atores e Casos de Uso



Cada ator é conectado aos casos de uso que ele pode executar por meio de uma **associação**.

Exemplo: Sistema de Consultas Médicas



Adicionando o Relacionamento «include»

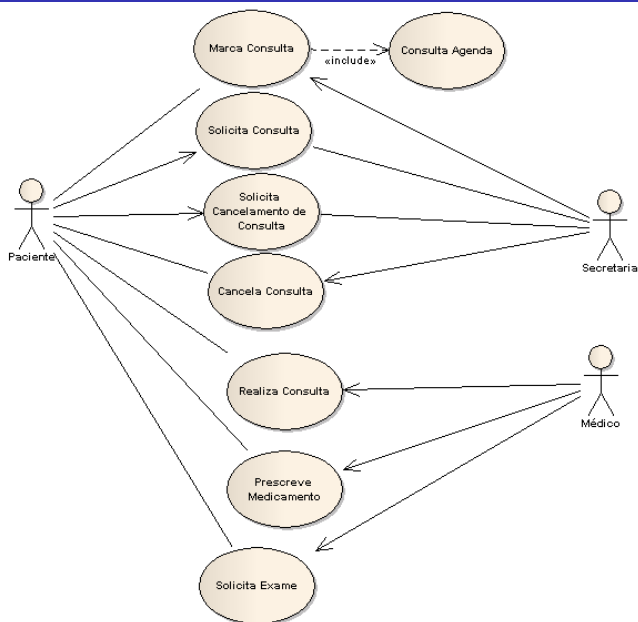
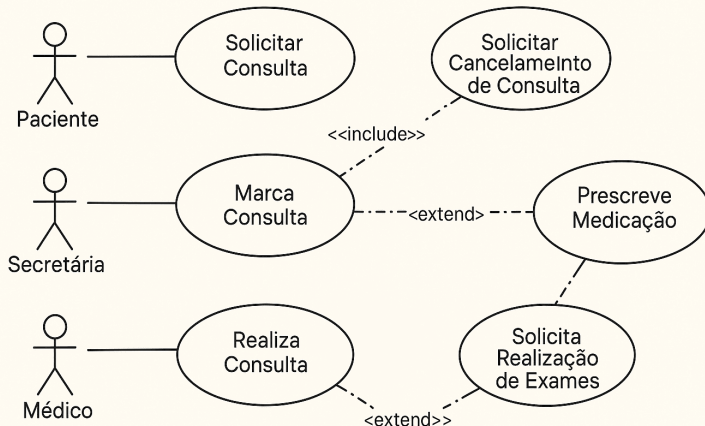


Diagrama de Caso de Uso



- Casos de uso são a forma padrão UML de descrever **interações entre atores e o sistema**.
- O diagrama de casos de uso oferece uma **visão geral** das funcionalidades; a descrição textual fornece o **detalhe**.
- Cada caso de uso pode englobar múltiplos **cenários**: caminho feliz, alternativo e de exceção.
- Casos de uso são mais úteis no **projeto de sistemas** do que na elicitação inicial de requisitos com leigos.
- Cada caso de uso deve ser documentado com **pré-condições, fluxos e pós-condições**.



- O **Documento de Requisitos de Software (SRS/ERS)** é a declaração oficial do que o sistema deve implementar.
- É essencial em sistemas **terceirizados, críticos** ou desenvolvidos por múltiplas equipes.
- Deve ser **organizado** para que clientes e desenvolvedores possam usá-lo.
- A **estrutura IEEE** inclui: prefácio, introdução, glossário, requisitos de usuário, arquitetura, especificação de sistema, modelos, evolução e apêndices.
- O documento tem usuários **diversificados**: clientes, gerentes, engenheiros, testadores e mantenedores.



Pontos-Chave: Validação de Requisitos

- Validação verifica se os requisitos definem o sistema que o cliente **realmente quer**.
- As cinco conferências essenciais: **validade, consistência, completude, realismo e verificabilidade**.
- Três técnicas principais: **revisões de requisitos, prototipação e geração de casos de teste**.
- Erros de requisitos descobertos **após a implementação** custam exponencialmente mais do que os descobertos cedo.
- É **raro** encontrar todos os problemas durante a validação — mais alterações serão necessárias.



- Mudanças em requisitos são **inevitáveis** — o ambiente de negócio, a tecnologia e os stakeholders mudam.
- Requisitos **duradouros** mudam lentamente; requisitos **voláteis** mudam frequentemente.
- O processo formal de gerenciamento de mudança tem três estágios: **análise do problema, análise de custo e implementação**.
- A **rastreabilidade** é essencial para avaliar o impacto de mudanças.
- Evitar modificar o sistema sem antes **documentar a mudança** nos requisitos.



O Ciclo Completo: Da Elicitação ao Gerenciamento

- 1 **Elicitação:** Descobrir o que os stakeholders precisam por meio de entrevistas, etnografia e cenários.
- 2 **Especificação:** Documentar em casos de uso, descrições estruturadas e no documento SRS.
- 3 **Validação:** Revisões, prototipação e geração de casos de teste para confirmar que os requisitos estão corretos.
- 4 **Gerenciamento:** Controlar mudanças, manter rastreabilidade e atualizar o documento de requisitos.

Regra de Ouro

Investir tempo na **qualidade dos requisitos** é sempre mais barato do que corrigir o sistema após a entrega.



- **Diagramas de Classes:** Os substantivos identificados nos fluxos dos casos de uso se tornam as **classes** do modelo de domínio.
- **Diagramas de Sequência:** Detalham como os objetos **colaboram** para realizar cada caso de uso.
- **Diagramas de Estado:** Mostram como o sistema **muda de estado** ao longo dos fluxos de cada caso de uso.
- **Padrões de Projeto:** Surgem como soluções recorrentes para problemas de design identificados durante a análise dos casos de uso.



Referências Bibliográficas

-  SOMMERVILLE, Ian. *Engenharia de Software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Cap. 4 (Engenharia de Requisitos), seções 4.4.3, 4.4.4, 4.5 e 4.6.
-  JACOBSON, I.; CHRISTERSON, M.; JONSSON, P.; OVERGAARD, G. *Object-Oriented Software Engineering*. Wokingham, UK: Addison-Wesley, 1993.
-  STEVENS, P.; POOLEY, R. *Using UML: Software Engineering with Objects and Components*. 2. ed. Harlow, UK: Addison-Wesley, 2006.
-  ROBERTSON, S.; ROBERTSON, J. *Mastering the Requirements Process*. 3. ed. Boston: Addison-Wesley, 2013.
-  IEEE. *IEEE Recommended Practice for Software Requirements Specifications*. IEEE Software Engineering Standards Collection. Los Alamitos, CA: IEEE Computer Society Press, 1998.



Dúvidas e Discussões

Leonardo S. Victorio

`leonardo.victorio@ufms.br`

Faculdade de Computação — UFMS

“A engenharia de requisitos é a fundação de todo o desenvolvimento de software.”

— Sommerville, 2011

